

14: Computer Arithmetic

– Multipliers

Unsigned binary multiplication

- Just like decimal long multiplication
 - Shift and conditional add product terms
 - Example 13×5

1101 ← Multiplicand
× 0101 ← Multiplier

1101
0000
1101
0000

01000001 ← Product

The diagram illustrates the step-by-step process of binary multiplication. The multiplicand 1101 is shifted left by the multiplier bits 0101. The first shift (multiplier bit 1) produces the product term 1101. The second shift (multiplier bit 0) produces the product term 0000. The third shift (multiplier bit 1) produces the product term 1101, shifted one position to the left. The fourth shift (multiplier bit 0) produces the product term 0000, shifted two positions to the left. The final product is the sum of these shifted terms: 01000001.

Unsigned binary multiplication (contd.)

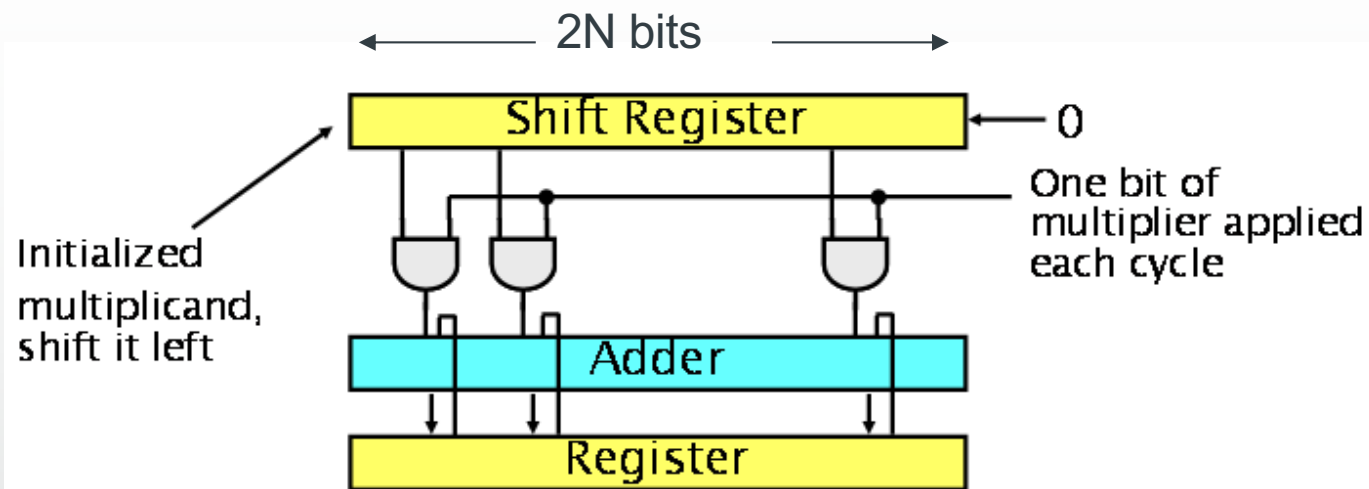
- Actually easier than decimal long multiplication
 - Because digits are either 1 or 0 – multiplication same as bit by bit logical AND of multiplier and multiplicand
 - It is the subsequent addition of partial products that takes effort
- If length of the multiplicand is N-bit and length of the multiplier is R-bit then
length of the product $P = N + R$
- In practice – a running sum of product term is made starting from the LSB
 - Rather than shifting the product left, the accumulated product is shifted right

Unsigned binary multiplication (contd.)

- In serial multiplication this iterative operation is executed successively in time
 - The multiplier is also shifted right on each step
- In parallel multiplication, the iterative operation is performed by an **array** of conditional adders
 - Successive right shift of accumulated product implemented by wiring between each stage

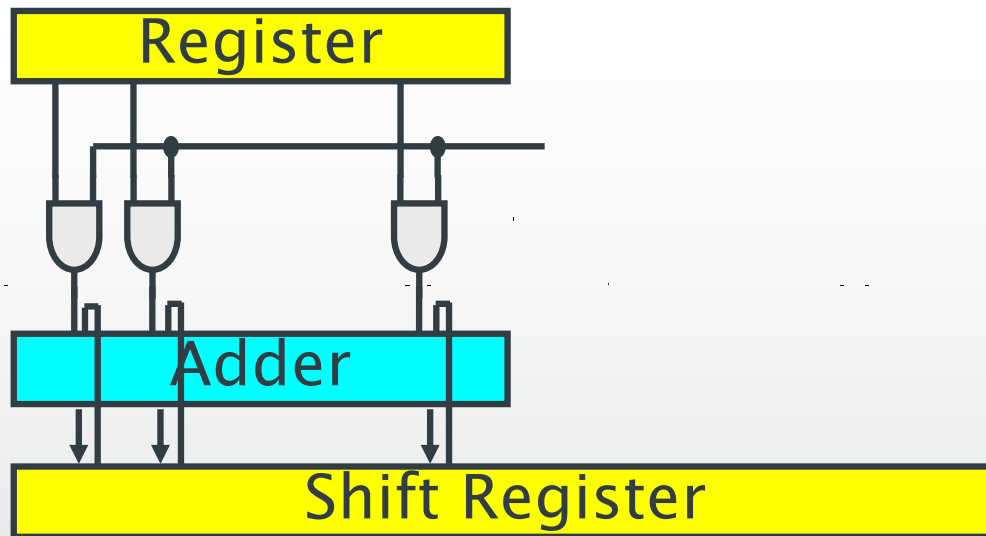
Sequential multiplier

- Shift register
 - Originally holds the multiplicand
 - Shifts it left for each of the partial products
- One bit of the multiplier is presented at a time to the AND gates



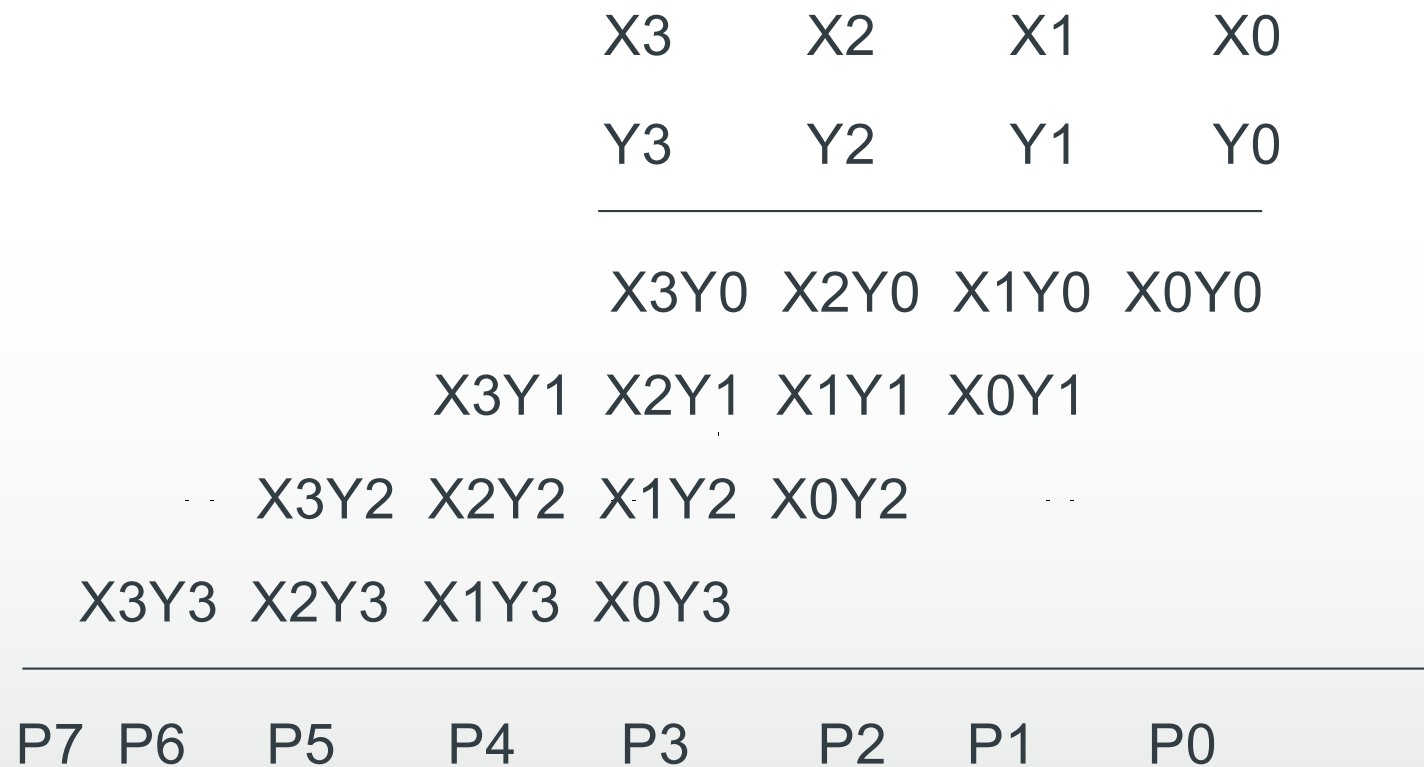
Sequential multiplier: resource requirement

- 2N-bit adder and 2N-bit register
- A better design
 - Shift result register to **right**: use N AND gate and N-bit adder

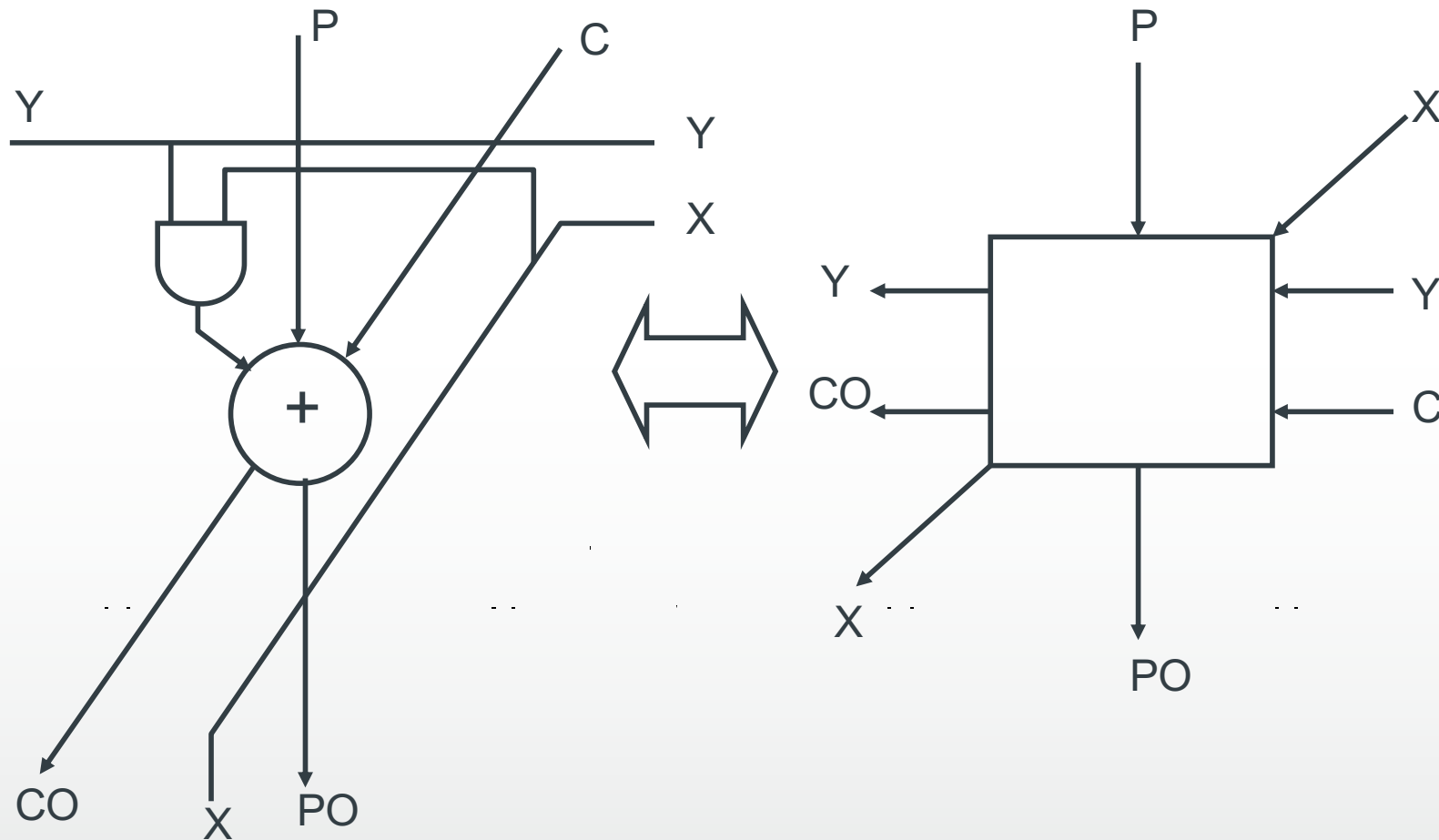


Parallel array multiplier

- 4-bit example



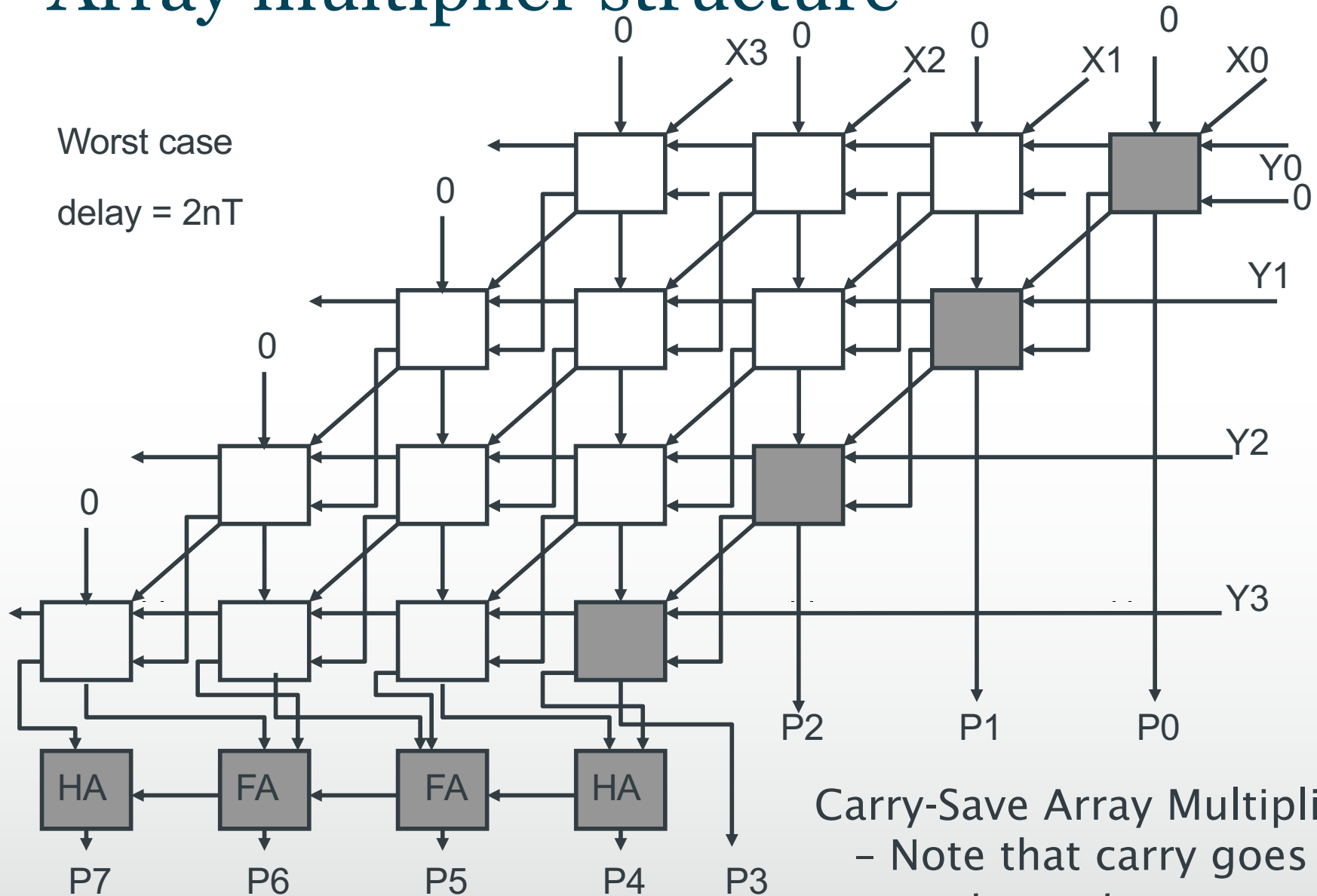
Array multiplier cell



Array multiplier structure

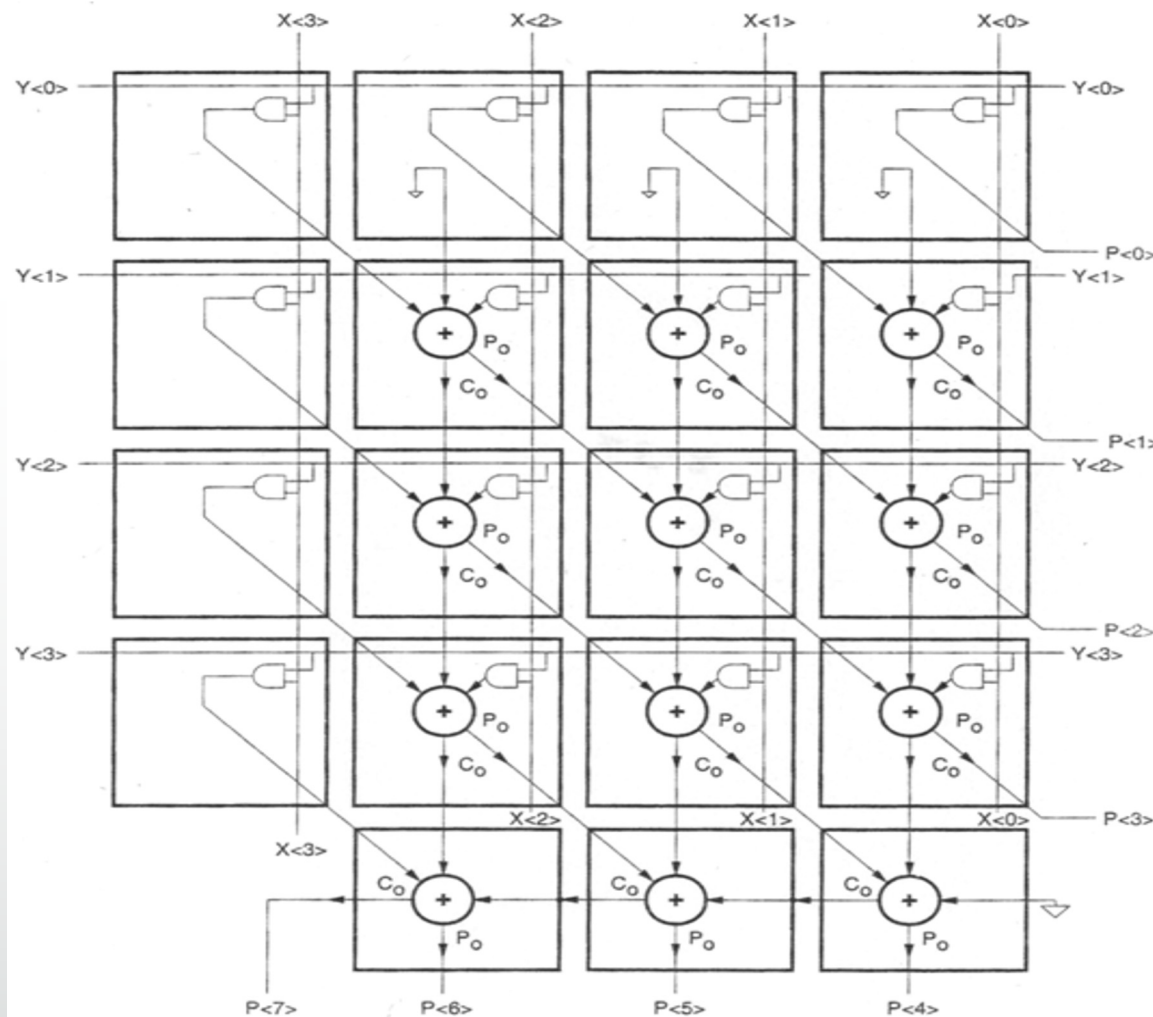
Worst case

delay = $2nT$



Carry-Save Array Multiplier
– Note that carry goes
to next row

Array multiplier structure: further improvement



Braun Array

Note that some cells are just multipliers (AND); some cells are just adders; some cells are half-adders ($c_i = 0$)

Unsigned binary multiplication: summary

- At each stage, LSB drops out to form part of partial product
 - The accumulating product is shifted right one place by wiring between each stage
- Using CPA settling time of the multiplier is given by the sum of:
 - 1 product AND gate delay
 - 1-input carry propagation delay
 - $N_d - 1$ carry-carry propagation delay
 - $N_r - 1$ input-sum propagation delay
- $N \times N$ multiplier requires $N(N-2)$ full adders, N half adders and N^2 AND gates

Signed binary multiplication

- Using 2's complement arithmetic signed multiplication is easy! (Don't believe this.)
- In order to deal with sign multiplicand: the sign bit of the partial product must be extended – e. g. -6×5

$$\begin{array}{rcll} & 1010 & \longleftarrow & -6 \\ & \times 0101 & \longleftarrow & 5 \\ \hline \text{Sign extension} \longrightarrow & 11111010 & \longleftarrow & \text{2's complement of 6} \\ & 0000000 & & \\ & 111010 & & \\ & 00000 & & \\ \hline & 11100010 & \longleftarrow & \text{2's complement of 30} \end{array}$$

Signed binary multiplication (contd.)

- In order to deal with signed multiplier: final conditional addition is replaced by a conditional subtraction
 - Because weight of the MSB of a 2's complement number is negative
 - Example 6×-5

$$\begin{array}{r} 0110 \quad \leftarrow 6 \\ \times 1011 \quad \leftarrow -5 \\ \hline 00000110 \\ 0000110 \\ 000000 \\ \hline 11010 \quad \leftarrow \text{2's complement of } 6 \\ \hline 11100010 \quad \leftarrow \text{2's complement of } 30 \end{array}$$

Booth's algorithm

- Alternative way to deal with signed multiplication
- Expects the current multiplier bit r_i as MSB and so it is treated as negative
- If later in the process, when dealing with the next most significant bit r_{i+1} it turns out to be wrong then a correction is made by adding twice the multiplicand

Booth's algorithm (contd.)

For i^{th} bit of the multiplier the algorithm is

r_i	r_{i-1}	Action
0	0	Add zero (do nothing)
1	0	Subtract multiplicand
0	1	Add multiplicand (correction)
1	1	Add zero (do nothing)

Since accumulated product is shifted once to the right,
correction addition is twice the previous subtraction

Booth's algorithm: example

$$\begin{array}{rcl} 0110 & \longleftarrow & 6 \\ \times 1011 & \longleftarrow & -5 \\ \hline 11111010 & \longleftarrow & \text{2's complement of 6 (bit } r_{-1} \text{ is assumed to be 0)} \\ 0000000 & \longleftarrow & \text{Add zero} \\ 000110 & \longleftarrow & \text{Add 6} \\ 11010 & \longleftarrow & \text{Subtract 6} \\ \hline 11100010 & \longleftarrow & \text{2's complement of 30} \end{array}$$

In serial multiplication Booth's algorithm has the advantage that it is not necessary to know when we come to the MSB and switch to a subtraction

Booth's algorithm: general comments

- For serial multiplication there is no significant difference in cost of implementation
- For parallel multiplication Booth's algorithm is more expensive
- Multiplication speed can be improved by processing two bits at a time using the modified Booth's algorithm